

```

condition 1 : sequence 1
condition 2 : sequence 2
...
condition n : sequence n
OTHERS:
default sequence

```

ENDCASE

REPEAT-UNTIL: This is quite akin to WHILE, but here the operation is performed at the bottom of the loop.

REPEAT

```
sequence
```

UNTIL condition

FOR: We employ FOR and ENDFOR in our algorithm for iterating the code block for a specific number of times. It is also called the “counting” loop.

FOR iteration bounds

```
sequence
```

ENDFOR

EXCEPTION HANDLING: The following code elucidates this.

BEGIN

```
statements
```

EXCEPTION

```
WHEN exception type
```

```
statements to handle exception
```

```
WHEN another exception type
```

```
statements to handle exception
```

END

You can also utilise methods like NESTED CONSTRUCTS, INVOKING SUBPROCEDURES to construct elegant and powerful algorithms.

We can define seven golden rules for writing algorithms:

- Use good code and good English.
- Ignore unnecessary details
- Take advantage of programming shorthand
- Be context specific
- Don't lose sight of the base model
- Always check for balance (can be easily implemented in a language)

Implementation

Let's write the code that produces a random number. Random number generation is an important tool in

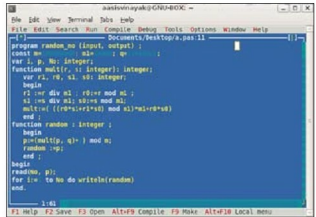


Figure 1: The free Pascal compiler

simulation building and scientific computing. In order to elucidate the implementation of the algorithm, I have written the code in Pascal.

```

program random_no(input, output);
const m=100000000; m1=10000; q=31415821;
var i, p, No: Integer;
function mult(r, s: integer): integer;
var r1, r0, s1, s0: integer;
begin
r1:=r div m1; r0:=r mod m1;
s1:=s div m1; s0:=s mod m1;
mult:=( (r0*s1+r1*s0) mod m1)*m1+r0*s0;
end;
function random : integer;
begin
p:=(mult(p, q)+1) mod m;
random:=p;
end;
begin
read(No, p);
for i:=1 to No do writeLn(random)
end.

```

It's assumed that you are skilled in grasping the meaning of simple codes. You can try this in your preferred Pascal compiler. (Figure 1 shows the code in Free Pascal Compiler, which I installed for trial.)

Euclid's Algorithm: A response to the queries

Many readers who tried the algorithms in the reference books (suggested in the last column) posed the question concerning the simplification of Euclid's algorithm and the base problem. As it was given as a problem, many of 'our passengers' got wedged between the lines!

For those who have not tried the exercises (if you don't have the book, you can find many online resources), here is a prologue to the problem:

Let a and b be integers, with $a \geq b \geq 0$. Using the